

Healthcare Appointment & Follow-up Manager

SYSTEM DESIGN DOCUMENT

Reliable appointment scheduling • AI-assisted consultation • Role-based healthcare workflow

Document purpose: Explain the system as an engineering design — including responsibilities, data ownership, invariants, request flows, concurrency, AI boundaries, failure handling, security, and traceability to the project requirements.

Source basis: Supplied project requirements + reviewed repository implementation. Where the requirements describe a target feature that is not currently evident in the reviewed repository, this document labels it as target/planned rather than claiming it as implemented.

1. Executive Summary

The Healthcare Appointment & Follow-up Manager is designed as a modular Django healthcare platform covering the full appointment lifecycle: authentication, doctor discovery, schedule/leave management, slot availability, temporary slot holding, symptom collection, consultation, and AI-assisted summaries. The central architectural problem is not simply creating appointments; it is preserving appointment correctness when multiple requests compete for the same doctor/time slot while keeping AI and external integrations from becoming single points of failure.

Primary architectural invariant: At most one active appointment may occupy a given doctor/date/start-time slot. This invariant is protected through application logic and a database constraint rather than relying only on UI availability.

Core architectural principle: The appointment system must remain operational even if the LLM or an external integration is unavailable. AI is an assistive subsystem, not the source of truth for scheduling or clinical records.

2. Design Goals

Goal	Design response
Correct scheduling	Transactional booking, row locking and database-level active-slot uniqueness.
Clear domain boundaries	Separate Django apps for accounts, doctors, appointments and consultations.
Responsive availability	Dedicated JSON slot endpoint consumed through JavaScript fetch().
AI-assisted workflow	Dedicated LLM service with explicit pre-visit and post-visit operations.
Failure isolation	LLM errors are represented as recoverable states rather than being allowed to crash the core workflow.
Maintainability	Business logic is separated into services/models instead of concentrating everything in views.
Traceability	Requirement-to-component mapping makes the system auditable by a reviewer.

3. High-Level Architecture

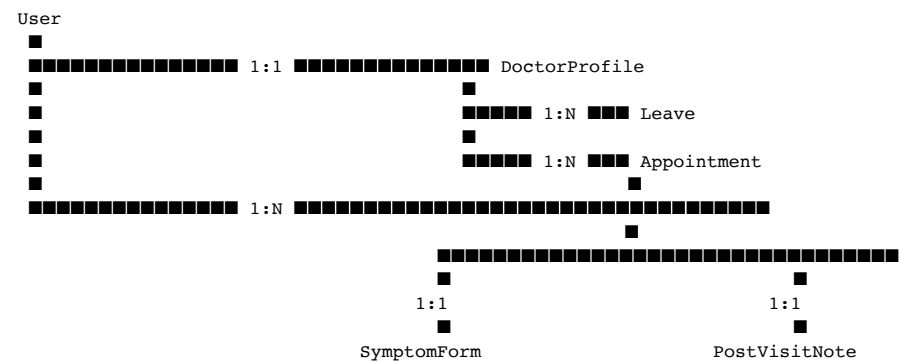


The current implementation uses Django's server-rendered approach. The browser receives HTML for the main workflow, while the slot picker uses a JSON endpoint for dynamic availability. The architecture intentionally keeps the scheduling domain independent from the AI provider.

4. Domain Decomposition

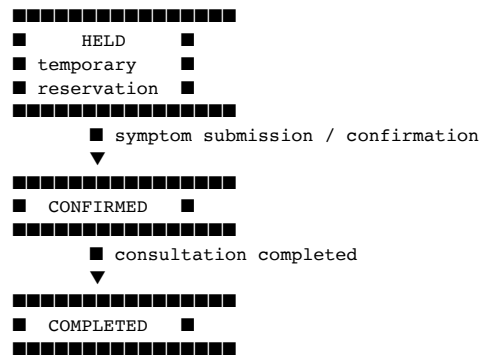
Component	Owns / coordinates	Why it exists
accounts	User identity, roles, authentication, access control	Keeps authorization rules separate from scheduling and consultation logic.
doctors	DoctorProfile, schedule, working days, leave, slot generation	Owns the definition of when a doctor can be booked.
appointments	Appointment lifecycle, availability, booking, cancellation	Owns the scheduling invariant and concurrency-sensitive business logic.
consultations	SymptomForm, PostVisitNote, LLM workflow	Owns information exchanged before and after the consultation.
notifications_app	Notification/integration boundary	Provides a natural extension point for email/calendar/background jobs.

5. Data Architecture



Entity	Key responsibility	Important design point
User	Identity + role	Supports patient / doctor / admin roles.
DoctorProfile	Professional + scheduling data	One-to-one with User; generates candidate slots.
Leave	Doctor unavailability	Unique doctor/date prevents duplicate leave records.
Appointment	Scheduling source of truth	Stores doctor, patient, date/time, state and hold expiration.
SymptomForm	Pre-visit information	One-to-one with appointment; includes AI-derived fields/status.
PostVisitNote	Post-visit information	One-to-one with appointment; includes clinical notes and AI-derived output.

6. Appointment State Model

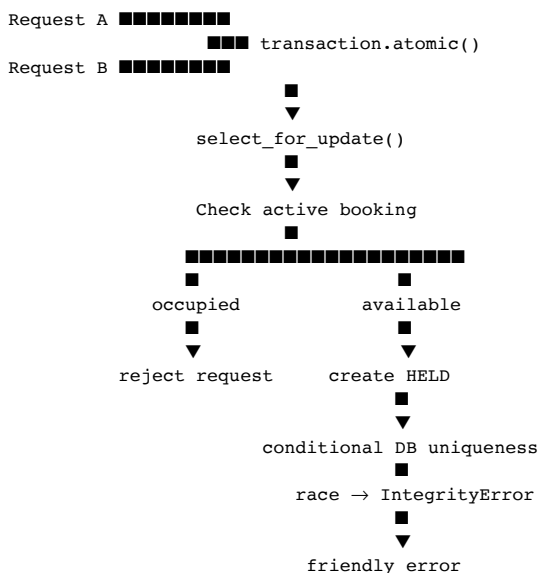


HELD / CONFIRMED CANCELLED
 HELD / CONFIRMED LEAVE_CANCELLED

The explicit status model makes the lifecycle queryable and prevents the system from treating every appointment row as an equally active booking.

7. Concurrency & Double-Booking Design

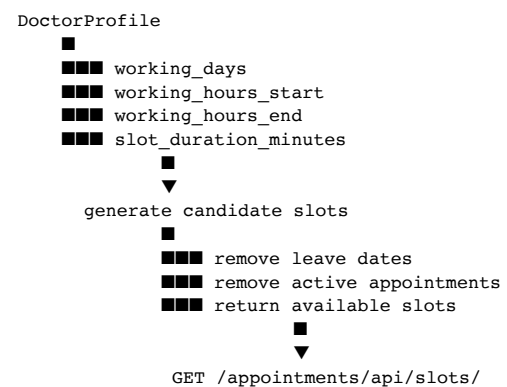
This is the most important consistency problem in the system. A UI-level availability check can become stale between the moment it is read and the moment the booking is written. Therefore, correctness is enforced at the transaction and database layers.



Invariant: For active states (held/confirmed), the same doctor/date/start_time combination cannot be occupied by two appointments.

Why both layers? Locking helps coordinate concurrent application requests; the database constraint is the final source-of-truth guard against a race that escapes the application check.

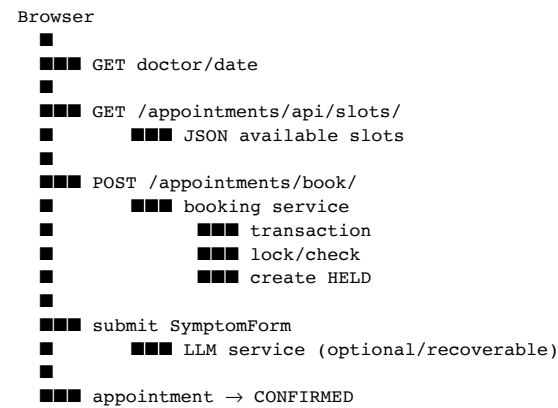
8. Slot Generation & Availability



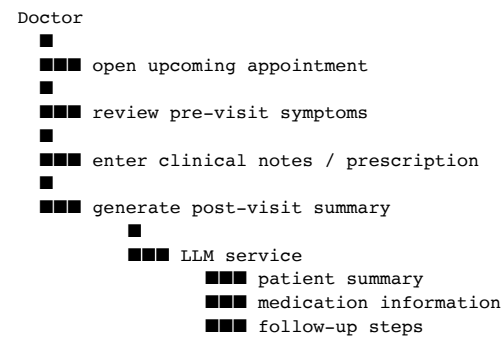
This creates a clean separation between **candidate availability** derived from the doctor's schedule and **occupied availability** derived from active appointments.

9. End-to-End Request Sequences

9.1 Patient Booking



9.2 Doctor Consultation



10. API Contract

Method	Endpoint	Purpose
GET	/appointments/api/slots/	Return currently available slots for a doctor/date.
GET	/appointments/doctors/	Search/list active doctors.
GET	/appointments/doctors/<id>/	View doctor details and slot context.
POST	/appointments/book/	Attempt to hold/book a selected slot.
GET	/appointments/mine/	Patient appointment history/upcoming appointments.
GET	/appointments/doctor/	Doctor appointment view.
POST	/appointments/<id>/cancel/	Cancel an appointment.

GET /appointments/api/slots/?doctor_id=2&date=2026-08-25

```
{
```

```

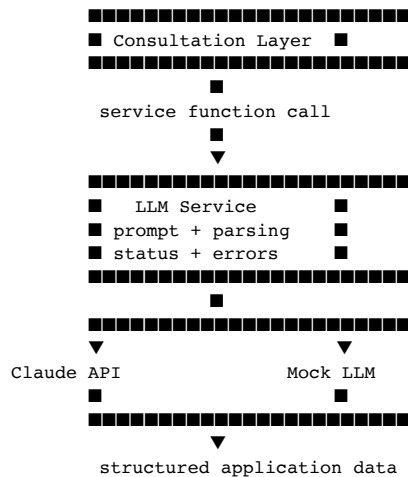
"doctor_id": 2,
"date": "2026-08-25",
"slots": ["09:00", "09:30", "10:00", "10:30"]
}

```

The slot API uses explicit input validation and meaningful HTTP status codes: 400 for invalid/missing request data and 404 when the requested doctor does not exist.

11. AI Subsystem Design

AI is deliberately placed behind a service boundary so the rest of the application does not need to know how prompts are constructed or which LLM provider is used.



AI operation	Input	Output
Pre-visit summary	Patient symptoms	Urgency, chief complaint, suggested questions
Post-visit summary	Clinical notes + prescription	Patient-friendly summary, medication information, follow-up steps

Failure boundary: LLM failure should update the LLM state/fallback path rather than corrupting the appointment state. Scheduling remains a database concern; AI remains an assistive concern.

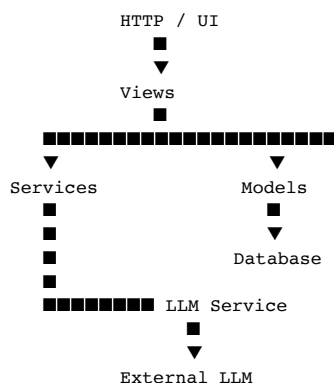
12. Security & Access Control Model

Layer	Control
Authentication	Django authentication with a custom User model.
Authorization	Role-aware access checks/decorators for patient, doctor and admin workflows.
Data integrity	Django model validation plus database constraints.
Secrets	Environment variables / .env.example pattern; secrets should not be committed.
AI boundary	LLM is not treated as an authority for appointment state or database consistency.
Healthcare safety	AI output should be treated as assistive information and reviewed by the appropriate professional.

13. Failure Modes & Recovery

Failure mode	Expected response
Concurrent booking	Lock/check + DB constraint; conflicting request receives a friendly slot-unavailable response.
Expired held slot	Hold expiry prevents an unfinished booking from permanently occupying a slot.
Invalid API parameters	Return 400 with a clear error message.
Unknown doctor	Return 404 rather than silently generating availability.
LLM/API failure	Record failure state and use a fallback path; do not make the external AI provider a scheduling dependency.
Doctor leave conflict	Leave is represented as a scheduling constraint and active booking logic must account for it.

14. Code Organization & Dependency Direction



Views coordinate HTTP concerns; services own business operations; models own persistence and invariants; the LLM service owns provider-specific AI behavior. This reduces coupling and makes individual components easier to test and evolve.

15. Requirement Traceability

The supplied requirements call for role-based authentication, doctor scheduling, leave handling, safe booking, slot holding, pre/post visit LLM support, email reliability, Google Calendar integration, background jobs, and a dynamic slot API. The table below separates what is represented in the reviewed implementation from target requirements that require final repository verification.

Requirement area	Architecture component	Reviewed status
Roles / authentication	accounts.User + access control	Implemented
Doctor scheduling	doctors.DoctorProfile	Implemented
Doctor leave	doctors.Leave	Implemented
Appointment lifecycle	appointments.Appointment	Implemented
Double-booking protection	booking service + transaction/locking + DB constraint	Implemented
Slot hold	Appointment held state + hold_expires_at	Implemented
Pre-visit AI	consultations.SymptomForm + LLM service	Implemented
Post-visit AI	consultations.PostVisitNote + LLM service	Implemented
Dynamic slot API	appointments slot JSON endpoint	Implemented
EmailLog / retry infrastructure	notifications/integration layer	Not evident in reviewed repository
MedicationReminder	notification/background layer	Not evident in reviewed repository
Google Calendar OAuth/event workflow	calendar integration layer	Not evident in reviewed repository
Background notification jobs	job/notification infrastructure	Not evident in reviewed repository

Accuracy rule: This document intentionally does not present target-only features as implemented. The final GitHub repository should be the source of truth for implementation claims.

16. Testing Strategy

The highest-value tests are those that validate invariants and failure behavior rather than only checking that individual pages render.

Test category	What should be proven
Concurrency	Two simultaneous requests for the same doctor/date/start_time cannot both create active appointments.
Slot availability	Booked/held slots are excluded from returned availability.
Hold lifecycle	Held slots expire and do not remain permanently blocked.
Authorization	Users cannot perform actions outside their role/ownership.
State transitions	Only valid appointment transitions are accepted.
LLM resilience	LLM failure produces a recoverable state rather than crashing the core workflow.
Leave handling	Doctor leave prevents incompatible booking behavior.

17. Observability & Operational Readiness

For a production deployment, the architecture should expose structured logs and metrics around booking conflicts, hold expiry, LLM latency/failure, email delivery, and calendar synchronization. These signals would make the system diagnosable without inspecting application state manually.

Signal	Why it matters
Booking conflict count	Detect demand spikes or scheduling contention.
Hold expiry count	Detect users abandoning the booking workflow.
LLM failure/latency	Measure dependency reliability and user-facing delay.
Notification failure/retry	Prevent silent communication failures.
Calendar sync errors	Detect divergence between application and external calendar state.

18. Scalability Path

The current SQLite implementation is appropriate for the specified development/demo environment. A production-oriented evolution would preserve the domain boundaries while replacing infrastructure components rather than rewriting the application.

```
Current demo
SQLite + Django
■
■■■ production DB → PostgreSQL
■■■ async jobs → Celery / worker + Redis
■■■ notifications → durable queue + retry policy
■■■ calendar → OAuth + event synchronization
■■■ observability → structured logs + metrics
```

This is an evolution path, not a claim that all production integrations are already present.

19. Design Trade-offs

Choice	Benefit	Trade-off
Server-rendered Django	Simple deployment and cohesive backend ownership.	Less decoupled than a separate SPA/API frontend.
SQLite	Very easy local setup and appropriate for the specified development environment.	Limited concurrency/scaling compared with a production relational server.
JSONField for AI outputs	Flexible for evolving semi-structured responses.	Less relationally constrained than normalized tables.
LLM service abstraction	Provider-specific logic is isolated.	Requires disciplined output validation and fallback behavior.
Database constraint + locking	Strong correctness guarantee.	Requires careful transaction boundaries and database-aware testing.

20. Reviewer Checklist

A reviewer should be able to verify the following without reading the entire repository:

- ✓ What problem is being solved? ✓ What are the core entities? ✓ Where is booking logic? ✓ How is double-booking prevented? ✓ What is the API contract? ✓ Where is AI integrated? ✓ What happens when AI fails? ✓ Which requirements are implemented versus planned?

21. Final Architecture Statement

The system is best understood as a **transaction-safe healthcare scheduling workflow with an AI-assisted consultation layer**. The database is the source of truth for appointment state; the service layer protects scheduling invariants; Django apps isolate domain responsibilities; and the LLM service provides assistive intelligence without becoming a dependency for core scheduling correctness.

One-line system design summary: “A modular Django healthcare platform that combines role-based appointment workflows, concurrency-safe slot booking, relational data integrity, and isolated AI-assisted consultation services.”

22. Source & Accuracy Note

This system design is grounded in the supplied project requirements and the reviewed repository structure/implementation. Requirement-specific target features that were not evident in the reviewed repository are explicitly marked as not evident rather than silently assumed to exist.